

POO e Tratamento de Arquivos

Exercício 1: Testar a construção de Arquivos

Objetivo

Testar o uso de arquivos.

Orientações

PARTE 1: manipulação de arquivos:

Etapa 1: Analise e teste os exemplos de manipulação de arquivos contidos na pasta exemplos.

Se estiver usando o Visual Studio 2022, apenas abra, compile e execute os projetos na seguinte ordem:

1.1) Mod_IO\exemplos\csharp\ProjetoArquivoExercicio1

- i) Observe que existe o arquivo nomeado customers-100.csv no mesmo diretório do arquivo Program.cs;

1.2) Mod_IO\exemplos\csharp\ProjetoSerializacaoArquivo

- ii) Analise as classes Pessoa.cs;
- iii) Analise a classe Program.cs;
- iv) O que significa “serializar” um objeto?
- v) Onde foi gerado o arquivo “pessoa.json”. Veja o diretório onde são gerados os códigos objetos do projeto.

Etapla 2: Crie o projeto com o nome ProjetoManipularArquivo e incorpore o seguinte “Main” no Program.cs:

```
using System.IO;

namespace ProjetoManipularArquivo{

    public class Program{

        static void Main(string[] args) {

            StreamWriter wr = new StreamWriter(@"D:\doc.txt", true);

            wr.WriteLine("Primeira linha");

            wr.WriteLine("Segunda linha");

            wr.Close();

            StreamReader rd = new StreamReader(@"D:\doc.txt");

            while (!rd.EndOfStream){

                string linha = rd.ReadLine();

                Console.WriteLine(linha);

            }

            rd.Close();

            Console.WriteLine("\n\nPressione qualquer tecla para sair...");

            Console.ReadKey(true);

        }

    }

}
```

i) Interprete a utilidade do programa;

Etapa 3: Crie o projeto com o nome ProjetoSeparaTokens e incorpore o seguinte “Main” no Program.cs:

```
using System.IO;

namespace ProjetoSeparaTokens{

    public class Program{

        static void Main(string[] args){

            string text = "Um exemplo de string para tokenização.";

            string[] tokens = text.Split(' ');

            foreach (string token in tokens){

                Console.WriteLine(token);

            }

            Console.WriteLine("\nDigite qualquer tecla para continuar...");

            Console.ReadKey();

        }

    }

}
```

i) Interprete a utilidade do programa;

Exercício 1) Crie um projeto com o nome ProjetoArquivoExercicio1 e faça o que se pede:

1.1) Acesse o link abaixo e baixe o arquivo que contém 100 registros de pessoas:
<https://www.datablist.com/learn/csv/download-sample-csv-files>;

1.2) Baixe e descompacte o arquivo customers-100.zip. Salve o arquivo customers-100.csv em um diretório com o nome sem espaço e sem acentuação.

Exemplo da máquina do Professor:

C:\00_academico\arquivo\customers-100.csv

1.3) Faça um programa para ler e exibir o conteúdo do arquivo na saída padrão, ou seja, na tela. Ao exibir o conteúdo na tela, dar uma pausa a cada 20 linhas. O programa deverá solicitar a intervenção do usuário, que ao clicar em qualquer

tecla, deverá continuar exibindo o conteúdo de forma paginada até atingir o final do arquivo.

Exercício 2) Faça um programa que receba um texto (*string*) e um *character* como parâmetro. Logo em seguida o programa deverá separar cada *token* da *string* usando o *character* informado como parâmetro.

PARTE 2: após estudar herança de objetos e o padrão singleton:

Exercício 3) Acesse o link:

<https://www.datablist.com/learn/csv/download-sample-csv-files>

Veja que existem diferentes tamanhos de arquivos (100, 1000, 10.000, ..., 2.000.000 de registros) contendo dados públicos de pessoas. Faça o download do arquivo que contém 100 registros, nomeado customers-100.csv.

2.1) Abra e analise o conteúdo do arquivo. Veja como o formato do arquivo está definido, como os registros estão separados, títulos das colunas, etc.

2.2) Crie a classe CustomerVO para encapsular os dados de um objeto do tipo Consumidor;

Obs: para testar os conceitos desta prática, basta criar os atributos para as três primeiras colunas;

2.3) Crie a classe DAOAb, que deve implementar:

i) padrão de projeto ***singleton***;

ii) A classe deve ter um atributo que guarda o nome do arquivo (do tipo string) a ser carregado na memória. Sugestão para o nome do arquivo: **nomeArquivo**;

Obs: como sugestão, você pode passar o nome do arquivo no método

```
public static DAOAb getInstance(string nomeArquivo){
```

```
    this.nomeArquivo = nomeArquivo;
```

```
    // restante da implementação do padrão singleton.
```

```
}
```

iii) Implemente na classe DAOAb o método que retorna todas as linhas do arquivo de nome **nomeArquivo** e tem a assinatura:

```
public List<string> obterTodosRegistros (){
```

```
// Abre o arquivo, lê todas as linhas, carrega cada linha em um objeto do tipo
List<string>, fecha o arquivo e retorna o objeto do tipo List<string>.
```

```
}
```

2.4) Crie a classe CustomerDAO, que deve implementar:

i) Herança com a classe DAOAb;

ii) deve implementar o padrão sigleton;

iii) deve implementar o método:

```
public List<CustomerVO> obterTodos(){
```

```
// Como sugestão você pode fazer uso do método genérico herdado da classe
DAOAb;
```

```
// Após retornar as linhas de um arquivo, fazer uso da “tokezinação” para gerar
outra coleção de objetos do tipo CustomerVO;
```

```
// retornar a coleção de objetos do tipo CustomerVO;
```

```
}
```

2.5) No método main do seu projeto, faça testes de DAOAb e CustomerDAO implementados;

Bom trabalho.